

Projektaufgabe 2

Phase 2 – Abschluss des Projekts (2.5 P)

Datenmanagement jenseits von Relationen

Gruppen Nummer (A12)

Zanotti Christian, 12528509

Wiesinger Martin, 11935667

Ottino David, 51841010

May 12, 2026

Dieses Reporting Template dient der Vorbereitung der Abgabe von Phase 2.

Alternativer Import für Ansatz 2 (0.5 Punkte)

Zeigen Sie das create table Statement für A oder B .

```
CREATE TABLE A_vec (  
    i INT NOT NULL,  
    row_vec DOUBLE PRECISION[]  
);
```

```
CREATE TABLE B_vec (  
    j INT NOT NULL,  
    col_vec DOUBLE PRECISION[]  
);
```

Zeigen Sie, wie die Matrizen A und B des Toy Examples für Ansatz 2 in der DB aussehen.

row	vec
0	{1.0, 0.0, 2.0, 0.0}
1	{0.0, 3.0, 0.0, 4.0}
2	{5.0, 0.0, 0.0, 6.0}

Table 1: Inhalt der Tabelle `toy_a_vec` (Matrix A)

col	vec
0	{7.0, 0.0, 1.0, 0.0}
1	{0.0, 9.0, 0.0, 3.0}
2	{8.0, 0.0, 2.0, 0.0}

Table 2: Inhalt der Tabelle `toy_b_vec` (Matrix B)

Ansatz 2 (0.5 Punkte):

Geben Sie den Code der Funktion `dotproduct()` bzw. Ihrer Lösung für Ansatz 2 an.

```
def create_functions(conn):
    with conn.cursor() as cur:
        # 1. Die langsame SQL-Variante (Ansatz 2a)
        cur.execute("""
            CREATE OR REPLACE FUNCTION dot_product_sql(vec1 DOUBLE PRECISION[],
            vec2 DOUBLE PRECISION[])
            RETURNS DOUBLE PRECISION AS $$
                SELECT COALESCE(SUM(v1 * v2), 0.0)
                FROM unnest(vec1, vec2) AS t(v1, v2);
            $$ LANGUAGE SQL;
        """)

        # 2. Die schnelle C-Variante (Ansatz 2b - Bonus)
        cur.execute("""
            CREATE OR REPLACE FUNCTION dot_product_c(DOUBLE PRECISION[],
            DOUBLE PRECISION[])
            RETURNS DOUBLE PRECISION
            AS '/var/lib/postgresql/extension_libs/dot_product.so',
            'dot_product_c'
            LANGUAGE C IMMUTABLE STRICT;
        """)
    conn.commit()

#include "postgres.h"
#include "utils/array.h"
#include "fmgr.h"

PG_MODULE_MAGIC;

PG_FUNCTION_INFO_V1(dot_product_c);

Datum
dot_product_c(PG_FUNCTION_ARGS)
{
    ArrayType *v1 = PG_GETARG_ARRAYTYPE_P(0);
    ArrayType *v2 = PG_GETARG_ARRAYTYPE_P(1);

    int n1 = ArrayGetNItems(ARR_NDIM(v1), ARR_DIMS(v1));
    int n2 = ArrayGetNItems(ARR_NDIM(v2), ARR_DIMS(v2));

    if (n1 != n2) PG_RETURN_FLOAT8(0.0);

    float8 *d1 = (float8 *) ARR_DATA_PTR(v1);
    float8 *d2 = (float8 *) ARR_DATA_PTR(v2);

    float8 sum = 0.0;
    for (int i = 0; i < n1; i++) {
        sum += d1[i] * d2[i];
    }

    PG_RETURN_FLOAT8(sum);
}

//cc -fPIC -I$(pg_config --includedir-server) -shared -o dot_product.so dot_produ
```

Zeigen Sie für das Toy Example, dass C korrekt berechnet wird (z.B. via Screenshot).

Zeile	Spalte	Ansatz 1 (Sparse)	Ansatz 2 (Vector)	Ergebnis (Soll)
0	0	9.0	9.0	9.0
0	1	-	0.0	0.0
0	2	12.0	12.0	12.0
1	0	-	0.0	0.0
1	1	39.0	39.0	39.0
1	2	-	0.0	0.0
2	0	35.0	35.0	35.0
2	1	18.0	18.0	18.0
2	2	40.0	40.0	40.0

Table 3: Vergleich der Berechnungsergebnisse des Toy Examples

Alle Ansätze liefern konsistente Ergebnisse, die exakt mit der theoretischen Matrixmultiplikation $C = A \times B$ übereinstimmen.

Benchmark Definition (0.5 P):

Parameter	Intervall	Kommentar
L	{8, 16, 32, 64}	Matrixdimensionen (Zweierpotenzen 2^3 bis 2^6)
S	{0.2, 0.5, 0.8}	Sparsity-Werte (von dünn besetzt bis fast voll)

Table 4: Getestete Matrixgrößen L und Sparsity-Werte S

Zur Evaluierung der Performance wurden quadratische Matrizen der Größen 8×8 bis 64×64 generiert. Um das Verhalten der verschiedenen Ansätze bei unterschiedlicher Datendichte zu untersuchen, wurden drei Sparsity-Stufen gewählt:

- **0.2 (Sparse):** Nur 20% der Matrix enthalten Werte ungleich Null.
- **0.5 (Neutral):** Hälfte der Matrix ist gefüllt.
- **0.8 (Dense):** Die Matrix ist zu 80% mit Werten besetzt.

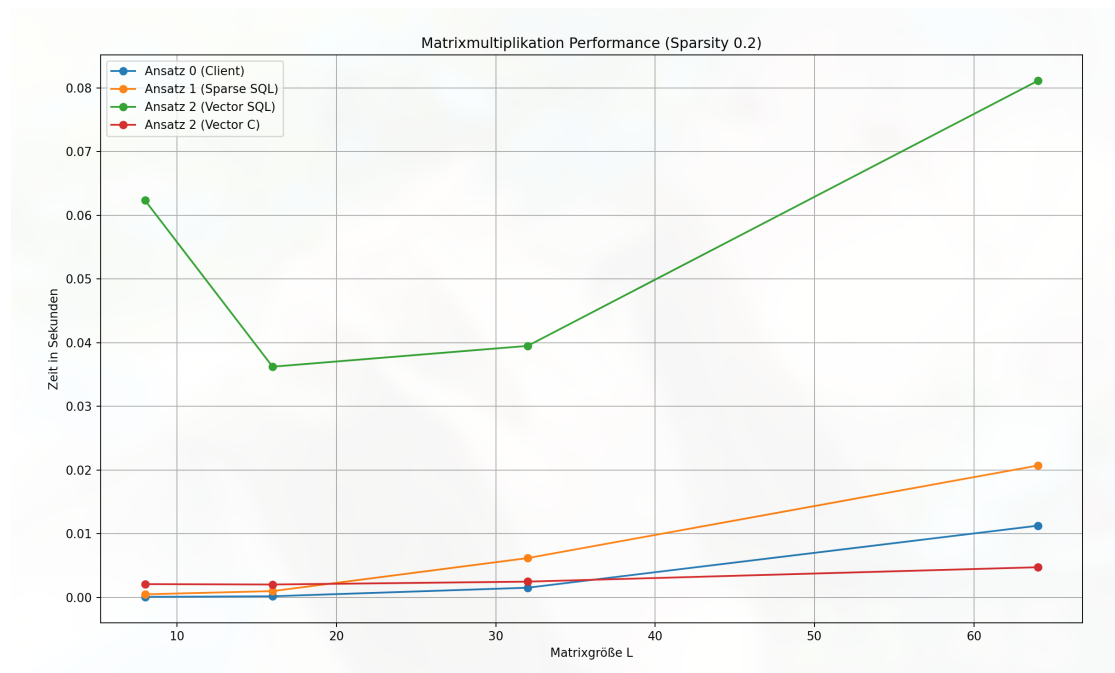


Figure 1: Fügen Sie Ihre Resultate als Grafik ein

Auswertung (0.5 P)

Stellen Sie Ihre Messergebnisse grafisch dar.

Zeitmanagement

Benötigte Zeit pro Person (nur Phase 2): **5**

References

Important: Reference your information sources!
 Remove this section if you use footnotes to reference your information sources.
